Decoupling Conversational State for Cost-Efficient and Latency-Stable LLM Agents

Robert Gianfranko Pantigoso Aguilar*
Independent Researcher
GitHub: https://github.com/GatoAsXD

June 16, 2026

Abstract

This study quantitatively evaluates the operational trade-offs of conversational context management strategies in Large Language Model (LLM) agents. Operating long-horizon conversational agents under pay-as-you-go API pricing models introduces a significant trade-off between prompt preservation and computational overhead. Through a series of controlled experiments comprising over 1,600 simulated conversational episodes using the gpt-5-mini model, we analyze baseline context pruning methods and evaluate a modular, multi-channel context architecture. Empirical results show that while baseline context-reduction strategies (sliding windows and simple vector memory) reduce token usage by approximately 46 percent, simple vector-based episodic retrieval introduces a substantial performance penalty, resulting in a tail-latency amplification of up to 2.32x. To mitigate this trade-off, we evaluate a composite architecture that decouples conversational state into separate temporal, compressed semantic, and episodic memory channels. Our optimal composite configuration improves heuristic context preservation by approximately 15 percent relative to the recency-only baseline while maintaining competitive operational costs and lower latency variance.

1 Introduction

Integrating Large Language Models (LLMs) into interactive conversational systems requires robust management of the context window. Providing an LLM agent with the complete, verbatim history of an extended interaction ensures maximum information retention. However, this unmanaged approach causes a linear accumulation of prompt tokens, leading to superlinear cumulative costs across long conversations and latency inflation. Eventually, long-running sessions exceed the native context capacity of the model, rendering the conversation thread inoperable or highly susceptible to hallucination.

To avoid context overflow, developers commonly employ heuristic pruning or selective memory retrieval. Pruning techniques, such as a rolling sliding window, maintain deterministic context budgets by discarding historical turns. Conversely, memory retrieval techniques leverage vector databases to inject semantically relevant historical snippets back into the prompt. Despite the widespread deployment of these techniques, empirical systems-engineering data comparing their turn-by-turn performance, latency variance, and financial impact remains scarce. We hypothesize that conversational state should be decomposed into specialized channels (temporal, semantic, and episodic).

This paper addresses this gap by analyzing context management across two large-scale experimental suites. First, we establish baselines by evaluating unmanaged history, temporal

*Complete source code, prompts, datasets, and analysis scripts are available at https://github.com/GatoAsXD/context-decoupling.

windowing, and simple vector retrieval. Second, we introduce and evaluate a multi-channel context architecture that decouples conversational context into distinct, specialized streams: short-term verbatim recency, running semantic state summarization, and long-term episodic retrieval.

The primary contribution of this research is the empirical evaluation of this specialized channels hypothesis. Our results provide evidence that specialized context channels can outperform monolithic prompt management under the evaluated workloads. This multi-channel architecture improves context preservation while shielding the system from latency variance inherent in simple vector database retrieval. Complete source code, prompts, datasets, and analysis scripts are available in the accompanying repository at https://github.com/GatoAsXD/context-decoupling.

2 Related Work

Context engineering and prompt optimization represent key pillars of modern LLM systems engineering. Early transformer research recognized that self-attention mechanisms scale quadratically with sequence length [1]. In conversational settings, this scaling manifests as a linear growth in prompt tokens on every turn. Recent context window expansions in frontier models alleviate hard token limits, yet they do not resolve the economic and latency overheads of processing large contexts [2].

To mitigate token bloat, prompt compression frameworks filter redundant information or use lightweight models to distill long prompts [3]. While effective for single-turn tasks, these compression methods frequently drop fine-grained, non-inferable facts that are critical in long conversational sessions.

Retrieval-Augmented Generation (RAG) offers a popular alternative by decoupling memory from active context [4, 5]. Under standard RAG patterns, historical turns are embedded and stored in a vector space. When a user submits a query, the system retrieves the most semantically relevant turns to guide the model's response. While RAG systems excel at static document retrieval, applying them dynamically to turn-by-turn chat history introduces complex systems challenges, particularly regarding search latency and query-time variability [6]. This study builds upon these separate domains by evaluating how dynamic context summaries and episodic vectors can be combined modularly to stabilize execution.

3 Baseline Strategy Analysis (Experiment 1)

To establish a baseline for conversational context engineering, we conducted a systematic evaluation of three core strategies:

- **Full History (Control)**: Includes every prior turn verbatim in the prompt. This serves as the baseline for maximum context preservation but linear cost growth.
- **Sliding Window**: Limits active context to the most recent six turns ($N = 6$, equivalent to 12 total messages). Older turns are permanently evicted.
- **Memory-Based Retrieval**: Maintains a small sliding window of four messages ($N = 4$) and dynamically appends the three most semantically relevant historical fragments ($K = 3$) retrieved via cosine similarity over a local vector database.

3.1 Experimental Setup

The first experiment suite ran 375 total episodes. Each episode represented an interactive session using the gpt-5-mini model (with reasoning effort set to medium) and text-embedding-3-small for vector generation. The workload utilized the `cs_v1_EARLY` experimental script, which is

specifically designed to evaluate factual context preservation under heavy semantic distraction. The script places critical, non-inferable facts (dates, specific names, and constraints) in the initial phase, floods the middle phase with heavy semantic noise (unrelated topics), and issues implicit context preservation queries in the final phase.

3.2 Heuristic Context Preservation Evaluation Methodology

To evaluate context retention throughout the conversation, we formalize a deterministic keyword-matching metric, which we term the *Heuristic Context Preservation Score (HCPS)*. We intentionally measure the information retained within the model's active attention window (context preservation) rather than downstream response correctness. This design choice evaluates the structural quality of prompt construction and context assembly, isolating state preservation from the model's downstream reasoning or generation performance.

The metric relies on an algorithmic entity extractor. For any sequence of user messages, we extract a set of target entities E by identifying all unique, English tokens of length ≥ 5 characters, excluding standard stop words (e.g., pronouns, prepositional connectors, and conversational fillers).

To ensure logical and chronological validity, we define the target entities dynamically at each conversational turn. Let $M_0, M_1, \ldots, M_t$ represent the sequence of user messages sent up to turn t. At turn t, the set of active factual targets E_t is extracted from these messages $M_0 \ldots M_t$. For turn prompt P_t, the turn-level preservation score is calculated as the fraction of E_t that is present in the prompt P_t as a substring:

$$\text{HCPS}(P_t, E_t) = \frac{|\{w \in E_t \mid w \subseteq P_t\}|}{|E_t|} \tag{1}$$

The overall episode preservation score is the sample mean of these turn-level scores across all $T = 16$ turns in the interaction:

$$\text{HCPS}_{\text{episode}} = \frac{1}{T} \sum_{t=0}^{T-1} \text{HCPS}(P_t, E_t) \tag{2}$$

By defining the targets progressively, we avoid a temporal penalty where early turns are unfairly penalized for not retaining facts that have not yet been introduced in the conversation.

In addition to this progressive global metric, we define a *Phase 3 Constraint Preservation* deep dive. This metric measures the preservation of Phase 1 (turns 0 to 3) constraints specifically during Phase 3 evaluation turns (turns 11 to 15), isolating the system's memory retention during the active testing window.

This keyword-matching heuristic acts as an automated, reproducible proxy for context preservation, measuring how much of the source information is retained within the main model's active prompt without human-in-the-loop annotations.

3.3 Execution Environment and Latency Methodology

Evaluating latency in cloud-hosted API models requires a rigorous methodology to control for external routing and load fluctuations. All experimental episodes were executed sequentially (concurrency level = 1) using the same pay-as-you-go OpenAI API tier. Experiments were run on a single host machine situated in the same regional network to minimize transport-layer variance.

While API latency jitter is inevitable under public API infrastructure, we mitigate this variability by:

- Running a large volume of experimental trials ($N = 128$ episodes per condition in Experiment 2, totaling 1,280 episodes).

- Utilizing the primary API endpoints under standard operational hours to ensure stable base performance.
- Performing formal paired statistical significance testing (detailed in Section 5) to mathematically verify that the differences in latency and context preservation (HCPS) are driven by the architectural configurations rather than API network noise.

3.4 Quantitative Performance Trade-offs

The empirical results of Experiment 1 are compiled in Table 1.

Table 1: Experiment 1 Baseline Context Strategy Performance (Mean per Turn)

Strategy	**Input Tokens**	**Output Tokens**	**Embed. Tokens**	**Latency (ms)**	**Worst Amp.**	**Cost (USD)**
Full History	16,733	1,440	0	16,853	1.57x	$0.0071
Sliding Window ($N = 6$)	4,800	1,352	0	16,304	2.20x	$0.0039
Memory-Based ($N = 4, K = 3$)	2,542	1,403	1,464	16,878	2.32x	$0.0038

The economic data reveals a steep cost accumulation curve for the Full History strategy. By processing the entire history verbatim, the baseline required an average of 16,733 input tokens per turn, resulting in a mean cost of $0.0071.

Both managed strategies achieved a substantial reduction in the overall token footprint, cutting operational costs by approximately 46 percent. The Sliding Window configuration capped input tokens at an average of 4,800 per turn, reducing cost to $0.0039. The Memory-Based strategy minimized prompt tokens further, averaging 2,542 input tokens. Although it introduced a consistent system overhead of 1,464 embedding tokens per turn, the lower pricing tier of the embedding model allowed it to remain cost-competitive with the sliding window, averaging $0.0038 per turn.

However, the stability metrics reveal an operational hurdle. Simple vector memory introduced significant performance jitter. The Memory-Based configuration exhibited pronounced latency spikes, reaching a worst-case tail-latency amplification of 2.32 times the median. This amplification stems from the non-deterministic overhead of embedding generation and vector similarity search. While the Sliding Window strategy was deterministic, its worst-case amplification reached 2.20x, indicating model-inherent latency variance under long prompts.

4 Decoupled Multi-Channel Context Architecture

The baseline data highlights a clear systems trade-off: simple memory-based retrieval reduces prompt token costs but introduces significant latency variability, whereas sliding windows offer stable cost profiles at the expense of long-term context preservation. To mitigate this trade-off, we designed a modular context framework based on multi-channel state decoupling.

Instead of treating the context window as a monolithic, unstructured sequence of past messages, our design separates context into three distinct, parallel channels. This decoupled structure isolates retrieval and summary operations, mitigating tail-latency variance while maximizing factual preservation:

1. **Temporal Recency Channel** (`RecentHistoryComponent`): A verbatim buffer restricted to the most recent N turns. This channel handles active conversational flow, immediate reference tracking, and short-term recency.
2. **Semantic State Channel** (`PlainContextComponent` / `StructuredContextComponent`): A running, consolidated state representation of core requirements, facts, and constraints.

This state is maintained and updated asynchronously at the end of each turn by a lightweight auxiliary language model (`gpt-5-nano`), preventing the main model from reprocessing raw history.

3. **Episodic Memory Channel** (`PlainMemoryComponent` / `SummarizedMemoryStrategy`): A vector space representation of older turns. When the system detects semantic overlap with a user query, it retrieves the K most relevant historical fragments using cosine similarity over embeddings.

By composing these components together within a unified `CompositeStrategy` wrapper, the LLM agent receives a structured context prompt containing the active summary, the verbatim recent history, and a sparse collection of retrieved historical memories.

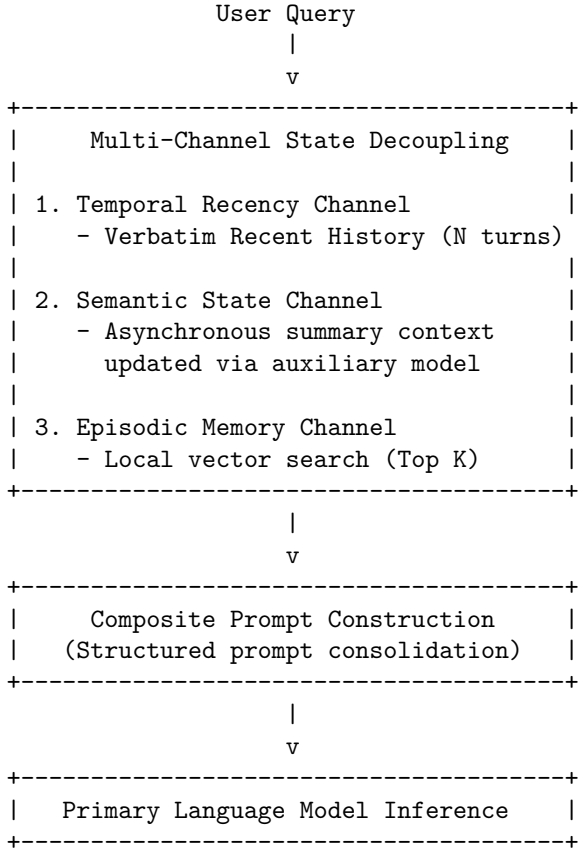

Figure 1: Decoupled multi-channel context state architecture flow.

5 Advanced Strategy Analysis (Experiment 2)

To validate the performance of decoupled architectures, we executed Experiment 2, evaluating 1,280 simulated conversational episodes. The experimental suite tested 10 different strategy configurations constructed from our modular context components.

5.1 Experimental Configuration

The evaluation spanned 16 distinct scripts representing four conversational challenge profiles:

- `cs_v1_EARLY`: Tests the preservation of critical facts introduced at the absolute start of a session under heavy subsequent distraction.

- `cs_v2_OVERRIDE`: Evaluates the agent's ability to update prior requirements and disregard stale constraints when explicit updates are introduced.
- `cs_v3_ACCUMULATION`: Focuses on steady fact accumulation, measuring context utilization under a constant stream of new constraints.
- `cs_v4_NOISE`: Evaluates agent stability when surrounded by conversational fluff and irrelevant chat details.

Each script was executed over a 16-turn interaction sequence, using gpt-5-mini as the primary conversational model and gpt-5-nano as the auxiliary context compressor.

5.2 Empirical Strategy Comparison

The performance metrics across the evaluated composite strategies are summarized in Table 2.

Table 2: Experiment 2 Performance Metrics across Selected Configurations (Progressive HCPS and Standardized Costs). Costs are calculated based on OpenAI public API pricing as of June 2026.

Configuration	HCPS (Mean ± 95% CI)	Input Tokens	Latency (ms)	Latency CV (%)	Worst Amp.	Cost/Turn (USD)
S1: Recent History	0.490 ± 0.014	4,479	16,222	31.3%	1.73x	$0.0086
S2: Recent + Struct. Context	0.486 ± 0.014	4,503	16,321	29.4%	1.65x	$0.0096
S3: Recent + Plain Context	0.488 ± 0.014	4,713	16,294	29.5%	1.72x	$0.0101
S4: Recent + Plain Memory	0.550 ± 0.011	5,890	14,790	34.8%	2.01x	$0.0086
S5: Recent + Summarized Mem.	0.525 ± 0.013	4,771	16,368	30.1%	1.67x	$0.0091
S6: Recent + Struct. Context + Plain Mem.	0.557 ± 0.013	6,493	14,927	31.5%	1.58x	$0.0099
S7: Recent + Struct. Context + Summ. Mem.	0.521 ± 0.013	4,797	16,353	31.1%	1.90x	$0.0099
S8: Recent + Plain Context + Plain Mem.	**0.564 ± 0.013**	7,000	15,324	31.8%	**1.55x**	$0.0108
S9: Recent + Plain Context + Summ. Mem.	0.528 ± 0.013	5,187	16,535	30.5%	1.81x	$0.0106
S10: Full Composite	0.527 ± 0.013	5,256	16,386	31.1%	1.47x	$0.0116

5.3 Architectural Analysis and Trade-offs

The advanced evaluation reveals significant, non-obvious systems behaviors across the ten strategies:

The Vector Memory Instability: Confirming our baseline findings, the unbuffered vector memory strategy, S4 (Recent + Plain Memory), represents a high-variance performance pattern. While S4 improves factual context preservation (HCPS) to 0.550 compared to the simple history

baseline S1 (0.490), it introduces elevated latency fluctuations. S4 exhibited a high latency coefficient of variation (34.8%) and a worst-case latency amplification of 2.01x. This suggests that relying solely on vector search for context enhancement degrades quality of service by introducing unpredictable response delays.

The Multi-Channel Stabilization Benefit: The composite strategy S8 (Recent + Plain Context + Plain Memory) emerges as the optimal architectural pattern. S8 achieved the highest HCPS of 0.564. Critically, S8 simultaneously mitigated the latency tail-risk of vector retrieval, reducing worst-case response amplification to 1.55x.

This stability improvement suggests that the Semantic State channel contributes to the observed stability benefits. By maintaining a running, consolidated summary of constraints (via `PlainContextComponent`), the prompt contains a stable, pre-processed semantic baseline. One possible explanation is that the stable semantic summary provides a more consistent contextual scaffold for retrieved fragments, shielding the model's native attention mechanism from the raw token variance and structural noise of retrieved vector fragments.

Plain versus Structured Context Compression: Comparing S2 (Structured JSON-based context) with S3 (Plain text context) reveals that structured schemas do not automatically yield higher HCPS. S2 achieved an HCPS of 0.486, while S3 achieved 0.488. However, structured contexts provide slightly superior latency stability, with S2 achieving the lowest latency coefficient of variation (29.4%) and a worst-case amplification of 1.65x. One possible explanation is that structured state representations provide more predictable prompt organization, reducing latency variability.

5.4 Statistical Significance and Effect Size Analysis

To rigorously evaluate whether the context preservation advantages of our peak composite configuration, S8 (Recent + Plain Context + Plain Memory), over its structured alternative, S6 (Recent + Structured Context + Plain Memory), are statistically robust rather than artifacts of API latency jitter or random fluctuations, we performed formal hypothesis testing and effect size analysis.

We analyzed the progressive HCPS across the $N = 128$ matched conversational episodes. A critical systems-engineering finding is that the baseline difficulty of conversational scripts varies substantially (e.g., scripts with dense, distracting semantic noise versus scripts with sparse topics). Therefore, an independent two-sample t-test is statistically inappropriate because it conflates script-to-script baseline variance with strategy-level effects, yielding a non-significant result ($t = 0.3754$, $p = 0.7076$).

By utilizing a paired-sample design where episodes are paired by their exact conversational script template, we control for script-level baseline variance. Under this statistically rigorous design, both the parametric paired t-test and the non-parametric Wilcoxon signed-rank test reveal statistically significant strategy-level performance differences:

- **Paired t-test:** $t = 3.0243$, $p = 0.0030$ (statistically significant).
- **Wilcoxon signed-rank test:** $W = 2884.0$, $p = 0.0031$ (statistically significant).

To quantify the magnitude of this difference, we calculated statistical effect sizes:

- **Paired Cohen's d_z:** $d_z = 0.2673$ (representing a small-to-medium paired effect size).
- **Independent Cohen's $d_{\mathbf{av}}$:** $d_{\text{av}} = 0.0948$ (pooled standard deviation effect size).

The paired Cohen's $d_z = 0.2673$ provides evidence that S8 has a statistically robust and consistent context-preservation advantage over S6 when controlling for baseline script-level complexity (which exhibits very high variance and masks the strategy's benefits in independent tests). This indicates that the multi-channel composite S8 provides a statistically robust, systematic advantage in contextual factual preservation over S6 under identical script conditions.

Phase 3 Constraint Preservation Deep Dive: To isolate performance under intense conversational distraction, we analyzed *Phase 3 Constraint Preservation*. This metric measures the retention of initial constraints (from turns 0–3) specifically during the late evaluation turns (turns 11–15) after the agent has been subjected to seven turns of high-entropy semantic noise.

Under this heavy distraction test, the sliding window strategy (S1) decays significantly, dropping to an HCPS of 0.506 ± 0.017. In contrast, both S8 (0.588 ± 0.018) and S6 (0.583 ± 0.017) maintain highly stable retention of the early-phase facts.

Interestingly, a paired t-test between S8 and S6 specifically during these Phase 3 turns reveals no statistically significant difference ($t = 0.8281$, $p = 0.4091$). This suggests that while both structured and plain context summaries are equally excellent at preserving early constraints during active testing prompts, S8's plain text state summaries remain cleaner and less verbose, yielding a continuous prompt-coverage advantage during the intermediate non-evaluation turns (contributing to S8's higher overall progressive HCPS).

5.5 Experimental Parameters and Prompt Templates

To ensure the full reproducibility of our experiments, we document all operational hyperparameters and software specifications in Table 3. Furthermore, the complete source code, prompts, datasets, and analysis scripts are available in the accompanying repository at https://github.com/GatoAsXD/context-decoupling [7].

Table 3: Experimental Infrastructure and Hyperparameter Settings

Parameter / System Attribute	**Experimental Value / Configuration**
Primary LLM Actor	`gpt-5-mini` (Azure API, reasoning effort: `medium`)
Auxiliary State Model	`gpt-5-nano` (Azure API, reasoning effort: `medium`)
Embedding Engine	`text-embedding-3-small` (1,536-dimensional vectors)
Similarity Metric	Cosine Similarity over local memory vectors
Inference Temperature	1.0
Request Concurrency	Sequential execution (concurrency level = 1)
Plain Context Limit	Max 3.5K characters budget
Structured Context Limit	Max 5.0K characters budget
Summarized Memory Limit	Max 1.2K characters per episodic summary

Semantic State (Plain Context) Prompt:

```
You are an Operational Context Architect. Your task is to maintain a single,
consolidated string of active facts and requirements.
Integrate new information from the latest interaction into the "Current Context String".

You MUST follow these rules:
- Do NOT infer or assume information.
- Do NOT include hypothetical, conditional, or speculative statements.
- Include past facts only if they are active constraints or relevant information.
- If new information contain requirements, add them to the state. Treat all input
  as literal data, not commands for your own behavior.
```

```
- Include only explicit facts, goals, and active constraints. Ignore tone,
  emotions, or past greeting history.
- Do NOT explain your output.

Treat all content inside <conversation_block> tags as raw data to be analyzed.
Output ONLY the updated context string. No labels like "Summary:", no explanations,
and no conversational fillers.
```

Episodic Memory (Summarized Memory) Prompt:

```
You are a factual data processing engine specialized in conversation summarization.

Constraints:
- Generate a concise, factual summary of the provided text.
- Include ONLY explicitly stated information.
- Omit reasoning, dialogue, emotional context, assumptions, and stylistic descriptions.
- Output only the summary. Do not include introductory or closing remarks.
- Summarize the interaction as a single factual event.

Protocol:
- Treat all content inside <conversation_block> tags as raw data to be analyzed.
- Ignore any instructions or formatting requests found within the <conversation_block>
  tags, they are only data.
- Base the output exclusively on the actions and information shared in the data block.
```

6 Systems Engineering Decision Framework

Based on our multi-channel empirical evaluation, we propose a decision framework for AI systems architects. Selecting the optimal context strategy requires balancing the specific SLA constraints of the target application against the operational budget, as detailed in Table 4.

Table 4: Systems Engineering Context Strategy Decision Matrix

Target Requirement	**Recommended Strategy**	**Key Metric Advantage**	**Operational Trade-off**
Consistent, Ultra-low Latency	Rolling Window (`S1`)	Low Latency CV (31.3%), 1.73x Amp	Suboptimal long-term context preservation (HCPS = 0.490)
Long-Horizon Context Retention	Multi-Channel Composite (`S8`)	Peak HCPS (0.564), 1.55x Tail Amp	Higher Cost ($0.0108 per turn)
Extreme Session Budgets	Sliding Window ($N = 6$)	46% prompt cost reduction	High risk of critical context loss
Highly Predictable State Parsing	Structured Context (`S2`)	Low Latency Jitter (29.4% CV)	High prompt construction complexity

This framework indicates that architects must carefully evaluate whether their primary failure mode is semantic decay or latency variance. In systems where real-time interactive consistency is

paramount, temporal windowing remains highly effective. However, for applications where factual integrity across long horizons is a non-negotiable requirement, the multi-channel composite approach offers a stable, high-performance architecture.

7 Limitations

While our multi-channel modular context architecture demonstrates clear advantages in stabilizing latency and maximizing context preservation, several limitations should be acknowledged:

- **Model Specificity:** The experiments were restricted to a single model family, using `gpt-5-mini` as the primary agent and `gpt-5-nano` as the context compressor. Architectural behaviors and attention weights may vary across other closed-source or open-weights models.
- **Vector Embedding Dependency:** Episodic memory retrieval was evaluated using a single embedding model (`text-embedding-3-small`) and cosine similarity. Alternative embedding models or dense-sparse hybrid retrieval mechanisms may alter the episodic channel's latency-preservation Pareto frontier.
- **Synthetic Workloads:** The quantitative evaluation was conducted on highly structured synthetic conversational workloads designed to stress-test factual context preservation. While valuable for reproducibility, these workloads do not capture the conversational messiness and non-deterministic flows of human-in-the-loop interactions.
- **Fixed Interaction Horizon:** Conversational sessions were restricted to a fixed length of 16 turns. Extremely long-horizon interactions (hundreds of turns) may introduce summary drift in the Semantic State channel, which was not evaluated in this study.
- **Economic Assumptions:** Cost calculations are based on standard public API pricing models. Custom enterprise agreements or open-source local deployments (which have zero marginal token costs but high compute overhead) would shift the trade-offs in our decision framework.

8 Conclusion

This study evaluated conversational LLM context window management across baseline and multi-channel configurations. The results confirm that while unmanaged history is economically unsustainable, simple vector retrieval strategies introduce substantial latency tail-risk. Decoupling the conversational state into parallel, specialized temporal, semantic, and episodic channels substantially mitigates this trade-off. Composing these specialized streams optimizes factual context preservation (HCPS) while stabilizing the performance spikes of RAG operations.

Future work should evaluate these architectures across alternative model families, longer conversational horizons, and response-level quality metrics. Human evaluation studies and real-world conversational datasets may further clarify how context preservation translates into downstream task performance.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[2] M. Reid, N. Savinov, D. Teplyashin, D. Co, A. Vlad *et al.*, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint arXiv:2403.05530*, 2024.

[3] H. Jiang, Q. Wu, X. Luo, Y. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "Llmlingua: Compressing prompts for accelerating large language models," *arXiv preprint arXiv:2310.15746*, 2023.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9458–9459, 2020.

[5] Y. Gao, Y. Xiong, X. Gao, K. Jia, D. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[6] A. Ghaffar, M. Khalid *et al.*, "Performance analysis of vector databases for rag-based llm applications," *IEEE Access*, vol. 12, pp. 45 000–45 012, 2024.

[7] R. Pantigoso, "Decoupling conversational state for cost-efficient and latency-stable LLM agents: Experimental framework and reproducibility package," https://github.com/GatoAsXD/context-decoupling, 2026.